2019 – CS241 - Sols

1. A new smartphone app is to be developed for registered users to hire cars. Registered users can use the app to find the location of available cars. The bank account of the user is debited automatically based on the time for which the car was used. The app will also be continually upgraded to enhance user experience. Outline at least 3 either social and ethical issues that the developer needs to take into account when developing this app.

When developing a smartphone app for hiring cars, developers need to consider various **social and ethical issues** to ensure the app is responsible, inclusive, and respects users' privacy and rights. Here are three key issues to take into account:

1. **Privacy and Data Security**: The app will collect personal information, such as user identification, location data, and banking details. Developers must ensure that user data is protected through strong encryption and secure storage. Ethical concerns arise if users' data is misused or if there is unauthorized access to sensitive information. Developers need to follow data protection regulations (such as GDPR) and provide transparent policies about how user data is collected, stored, and used.

2. **Fairness and Accessibility**: The app should be designed to be inclusive, ensuring that all users, including those with disabilities or those from different socioeconomic backgrounds, can use it easily. The developers must also ensure that the pricing model is transparent, fair, and non-discriminatory, preventing exploitative practices such as surge pricing during peak times or unfairly high charges for certain demographics.

3. **Environmental Impact and Sustainability**: The app encourages the use of rental cars, which can have environmental implications. Developers should consider promoting eco-friendly options, such as electric vehicles or car-sharing systems, to reduce the carbon footprint. Additionally, the app could provide users with information on the environmental impact of their travel choices and incentivize environmentally conscious decisions.

By addressing these issues, the developers can create an app that is not only user-friendly but also ethical and socially responsible.

2. Explain 2 benefits of regular contact between developers and clients during the software development process.

Regular communication between developers and clients during the software development process can lead to several benefits, including:

- **Faster development cycles**

Developers can identify and address issues early on, which can help save time and resources.

- **Improved customer satisfaction**

Clients are more likely to feel invested in the development process and develop a sense of trust with the development team.

3. During a one-day sale, an online shopping site became unresponsive when too many people attempted to visit the site at the same time. Describe the testing techniques that could have been used to prevent this issue from occurring.

To prevent the issue of the online shopping site becoming unresponsive during a one-day sale, several **performance testing techniques** could have been used:

1. **Load Testing**: This technique simulates the expected number of concurrent users on the site to check if it can handle the anticipated traffic without performance degradation. By testing the site under normal usage conditions, the team can ensure it remains responsive and stable during typical peak loads.
2. **Stress Testing**: Stress testing involves pushing the site beyond its expected capacity to identify the breaking points. By simulating more users or higher traffic volumes than the system can typically handle, developers can observe how the site behaves under extreme conditions and ensure that it fails gracefully or provides alerts when it reaches its capacity limit.
3. **Scalability Testing**: This technique assesses whether the system can handle increasing numbers of users over time by testing how the site behaves when more resources (e.g., servers, bandwidth) are added. It ensures the system is scalable and can handle a surge in traffic without crashing.
4. **Spike Testing**: Spike testing simulates a sudden and sharp increase in traffic, such as during the sale, to see if the system can recover quickly and maintain performance under unexpected spikes. This can help identify any weaknesses in the site's ability to handle sudden user influxes.
5. **Endurance Testing (Soak Testing)**: This testing checks how the system performs over extended periods of time under a sustained load. It helps ensure that the system can handle long-term usage, which is important for websites that experience high traffic over the course of the sale day.

By employing these testing techniques in advance, the development team can identify potential bottlenecks and ensure the site can handle large volumes of traffic without becoming unresponsive.

4. Describe the benefits of using encapsulation in object oriented programming.

Data Hiding: Encapsulation helps in hiding the internal details of an object. This means that the data (attributes) of a class are kept private, and access to them is provided through public methods (getters and setters). This prevents direct modification of data, ensuring that it can only be altered through controlled methods. This leads to better security and integrity of the data.

Control Over Data Access: With encapsulation, you can control how the data is accessed and modified. For example, you can provide only read-access or restrict modifications to certain conditions, ensuring that invalid states are not introduced into the object. This can help in implementing business rules and validations within the setter methods.

Security: Encapsulation hides the internal state of objects, preventing unauthorized access and manipulation.

5. Compare similarities and differences between storyboards and screen designs.

   Storyboards and screen designs both serve to visualize the user experience in an application, but they differ in purpose and detail. Storyboards focus on illustrating the sequence of actions or user interactions, showing how the user moves from one screen to another in a flow. They provide a high-level view of the user journey, emphasizing transitions and steps rather than specific visual elements. In contrast, screen designs concentrate on the individual layout and aesthetics of each screen, including details such as color schemes, typography, buttons, and overall design. They are highly detailed and offer a clear representation of what each screen will look like. While storyboards help map out the user's journey through the app, screen designs provide the blueprint for building each interface. Together, they ensure the design is both functional and visually appealing, with storyboards guiding the flow and screen designs providing the visual structure.

6. A software package is being developed for a retail store. The package will include separate modules for tracking stock, recording sales and maintaining staff details.
   a. Explain why a phased method of installation is appropriate for this system.
      A phased method of installation is appropriate for this retail store software system because it allows for a gradual transition from the old system to the new system. This approach minimizes the risk of disruption in daily operations. Each module (stock tracking, sales recording, and staff details) can be implemented and tested one at a time, ensuring that any issues are identified and addressed early without affecting the entire system. For instance, the stock tracking module could be implemented first, followed by the sales recording module and, lastly, the staff details module. Phased installation also allows users to become familiar with the system incrementally, making the learning curve more manageable.

   b. Describe how a system modelling tool can be used in the development of this software.
      A system modeling tool can be used to visually represent the components of the retail software system and their interactions. It helps in creating data flow diagrams (DFD), entity-relationship diagrams (ERD), and other models that define the system's architecture and behavior. This enables developers to plan the system's structure, identify potential issues in the design, and communicate effectively with stakeholders. For example, a DFD can represent how information flows between the modules (e.g., stock tracking and sales), helping developers understand dependencies and streamline the integration of each module.

   c. Describe how the team leader should manage the development of this software.
      The team leader should manage the software development by ensuring clear communication and setting realistic milestones. They must assign tasks to team members based on their expertise, ensure the modules are developed according to specifications, and regularly monitor progress. Regular meetings and reviews should be held to track the development of each module and address any challenges that arise. Additionally, the team leader must coordinate with other departments (such as sales and inventory) to ensure the software meets all user needs. Risk management and testing should also be prioritized to identify issues early and ensure that the software is reliable before the full rollout.

7. Describe 2 benefits of a software developer for using CASE tools.
   CASE tools offer several advantages, including improved productivity, enhanced collaboration, better quality assurance, automated documentation, standardization, and effective risk management.

   Improved Efficiency and Productivity: CASE (Computer-Aided Software Engineering) tools help automate many repetitive and time-consuming tasks involved in software development, such as code generation, documentation, and system modeling. By providing a streamlined approach to these tasks, CASE tools can significantly reduce manual effort, allowing software developers to focus on more critical aspects of the development process, like design and problem-solving. This leads to faster development cycles and higher productivity.

   Enhanced Quality and Consistency: CASE tools provide standardized templates, guidelines, and frameworks that help ensure consistency in the software development process. These tools promote adherence to best practices and coding standards, reducing the chances of errors or inconsistencies in the final software product. Additionally, built-in testing and debugging features help developers identify issues early in the development process, improving the overall quality and reliability of the software.

8. There are many different programming languages available. Outline TWO factors a developer should consider when choosing a programming language for developing a particular piece of software.

   Project Requirements and Functionality: The specific requirements of the project are crucial in determining which programming language to choose. For example, if the software needs to interact with hardware or perform real-time processing, languages like C or C++ might be preferred due to their low-level capabilities and performance. If the project involves web development, languages like JavaScript, HTML, and CSS are more suitable. The developer should assess the complexity of the project, the functionality needed, and whether certain libraries, frameworks, or features are available in specific languages to meet these needs efficiently.

   Development Time and Ease of Use: The time available for development and the ease of learning and using a programming language are important factors. Languages like Python, for instance, offer a simpler syntax and a wide range of libraries, making it ideal for rapid development. On the other hand, languages such as Java or C# may have a steeper learning curve but are suitable for larger, more complex systems. The developer should consider how quickly they can write, test, and deploy the software in the chosen language.

9. Explain polymorphism in relation to object oriented programming?
   Polymorphism in object-oriented programming (OOP) refers to the ability of different objects to respond to the same method call in a way that is specific to their respective types. It allows for methods to have different implementations depending on the object that is calling them, even if the method name is the same.

10. A pet shop has developed a software system to control its inventory. The following fragment of java code used for the system is based on the object oriented programming (OOP) paradigm. class

```
Pet {
        private int itemNumber;
        private double price;
        public Pet();
        public string supplier;
        public getPrice() {
        RETURN price;
        }
}
class Dog {
        public Dog();
        public string breed; public string name;
        public string colour;
        public Boolean male;
}
public Boolean microchipped;
        class Accessory {
        private int itemNumber;
        public Accessory();
        public string type;
        public double price;
}
```

a. Using examples from the above code, outline the differences between the private and public declarations within a class.

In object-oriented programming (OOP), **private** and **public** are access modifiers that control the visibility and accessibility of class members (variables and methods) from other classes.

I. **Private Declarations**:
   - **Private members** are only accessible within the same class where they are defined. They cannot be accessed directly from outside the class.
   - In the provided code, the itemNumber and price fields in the Pet class are declared as private, which means they cannot be accessed directly from outside the Pet class.
   - Example:

```
private int itemNumber;
private double price;
```

These fields can only be accessed within the Pet class or through public getter/setter methods.

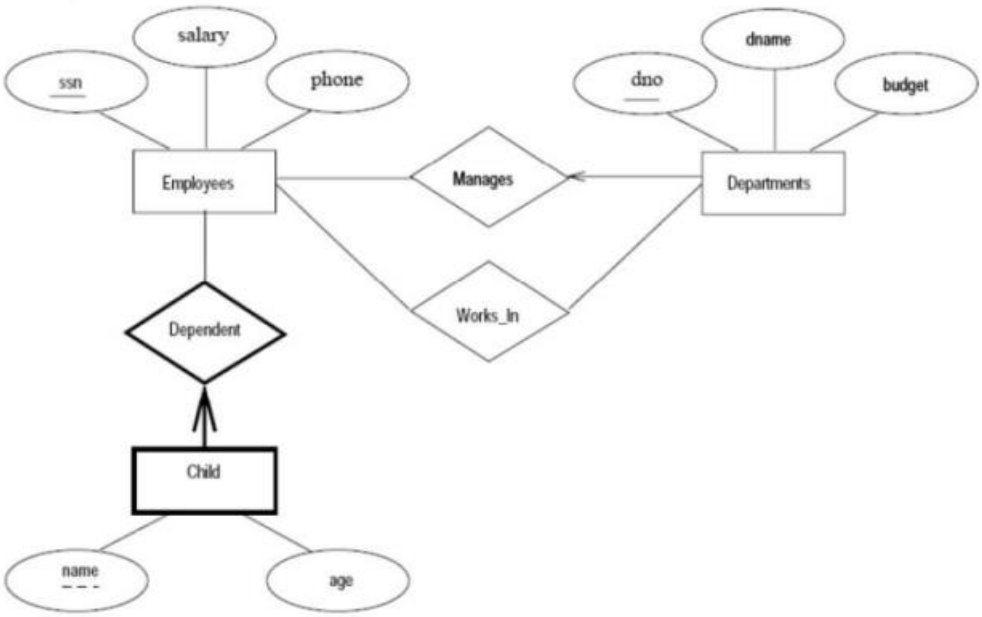
II. **Public Declarations**:
   - **Public members** are accessible from any other class. This means that their values can be read and modified directly from any code outside the class.
   - In the provided code, the supplier, breed, name, colour, male, and microchipped fields are declared as public, which makes them accessible from other classes.
   - Example:

```
public string supplier;
```

Other classes can directly access and modify supplier.

b. Draw a design class diagram showing the above objects and the inheritance relationship between the Pet and Dog objects.

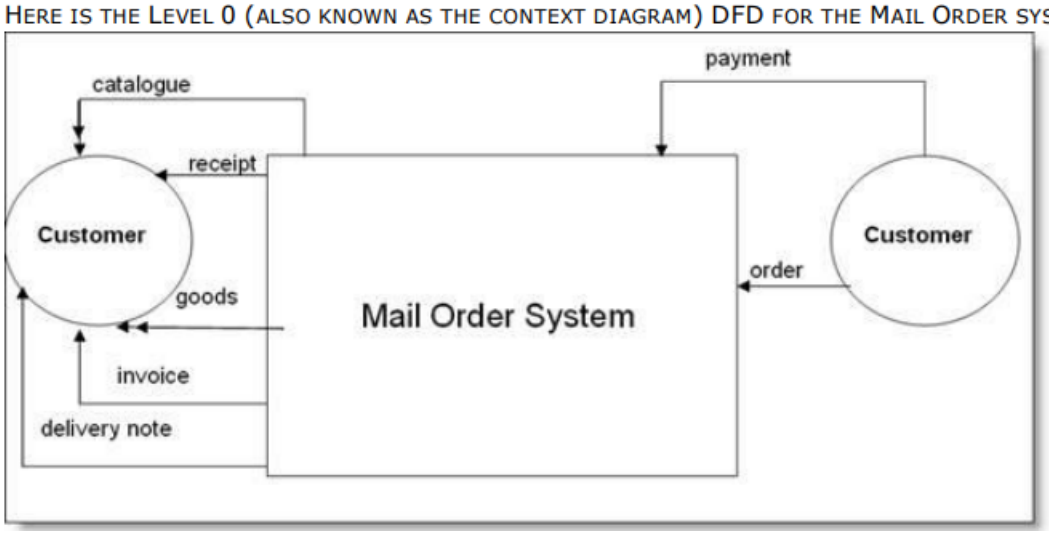
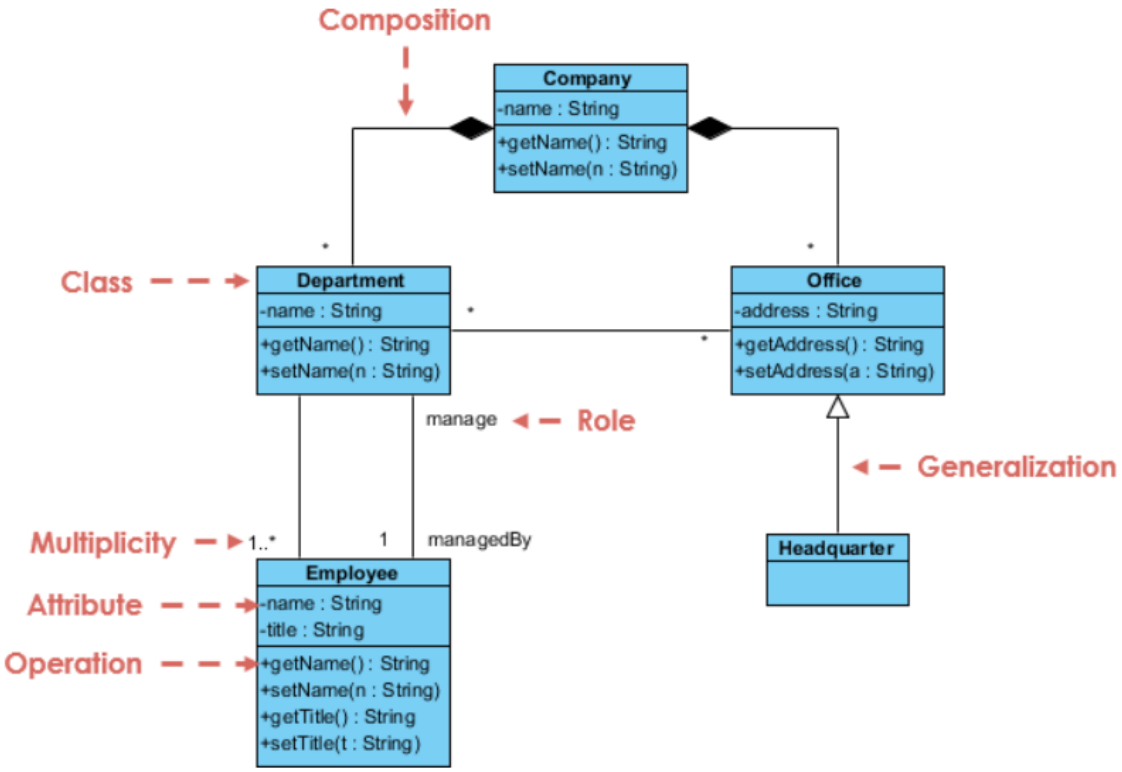
11. A company database needs to store information about employees (identified by ssn, with salary and phone as attributes), departments (identified by dno, with dname and budget as attributes), and children of employees (with name and age as attributes). Employees work in departments; each department is managed by an employee; a child must be identified uniquely by name when the parent (who is an employee; assume that only one parent works for the company) is known. We are not interested in information about a child once the parent leaves the company. Draw an ER diagram that captures this information.



12. You are given the details of a small mail order catalogue system that allows people to shop from home. When a customer receives the catalogue and wants to buy something, they can telephone, fax or email their order to the company. The company gets the order and sends the goods and an invoice. When the customer receives the goods with a delivery note, they send payment and receive a receipt for their payment. Draw a Level O DFD to reflect this Mail Order Catalogue System. SEE PDF

LEVEL 0 DFD:

HERE IS THE LEVEL 0 (ALSO KNOWN AS THE CONTEXT DIAGRAM) DFD FOR THE MAIL ORDER SYSTEM:



13. A company consist of departments. Departments are located in one or more offices. One office acts as a headquarter. Each department has a manager who is recruited from the set of employees. Draw a design class diagram which consists of all the classes in your system their attributes and operations, relationships between the classes, multiplicity specifications and other model elements that you find appropriate.